

Question 1

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load dataset and check for missing values
df = pd.read_csv("Dataset_Day14.csv")
print("\n Missing values in each column:\n")
print(df.isnull().sum())

# Drop missing rows if any (optional)
df.dropna(inplace=True)
print(f"\nData shape after removing missing values: {df.shape}")

# Extract numeric features
features = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']
X = df[features]

# Scale the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\n Features have been scaled using StandardScaler.")
```

Step 1: Missing values in each column:

Id	0
SepalLengthCm	0
SepalWidthCm	0
PetalLengthCm	0
PetalWidthCm	0
Species	0
dtype: int64	

Data shape after removing missing values: (150, 6)

Features have been scaled using StandardScaler.

Question 2

```
# 2. Fit DBSCAN with default values (eps=0.8, min_samples=4)
print("\n: Applying DBSCAN with eps=0.8, min_samples=4 ...")
dbscan_default = DBSCAN(eps=0.8, min_samples=4)
df['Cluster_default'] = dbscan_default.fit_predict(X_scaled)

# Show species distribution in clusters
print("\n Species distribution in default DBSCAN clusters:")
print(pd.crosstab(df['Cluster_default'], df['Species']))
```

```
: Applying DBSCAN with eps=0.8, min_samples=4 ...
```

```
Species distribution in default DBSCAN clusters:
```

Species	Iris-setosa	Iris-versicolor	Iris-virginica
Cluster_default			
-1	1	0	3
0	49	0	0
1	0	50	47

Question 4

```
# 3. Use Nearest Neighbors to find optimal eps
print("\n Plotting k-distance graph to estimate optimal eps")
neighbors = NearestNeighbors(n_neighbors=5)
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)
k_distance = np.sort(distances[:, 4])

# Plot k-distance graph
plt.figure(figsize=(8, 4))
plt.plot(k_distance)
plt.title("K-distance Graph to Determine Optimal eps")
plt.xlabel("Data Points Sorted")
plt.ylabel("5th Nearest Neighbor Distance")
plt.grid(True)
plt.show()
```

Question 3

Plotting k-distance graph to estimate optimal eps...

Step 4: Silhouette scores for min_samples from 3 to 9 (eps=0.8):

min_samples = 3, Silhouette Score = 0.5206

min_samples = 4, Silhouette Score = 0.5206

min_samples = 5, Silhouette Score = 0.5206

min_samples = 6, Silhouette Score = 0.5206

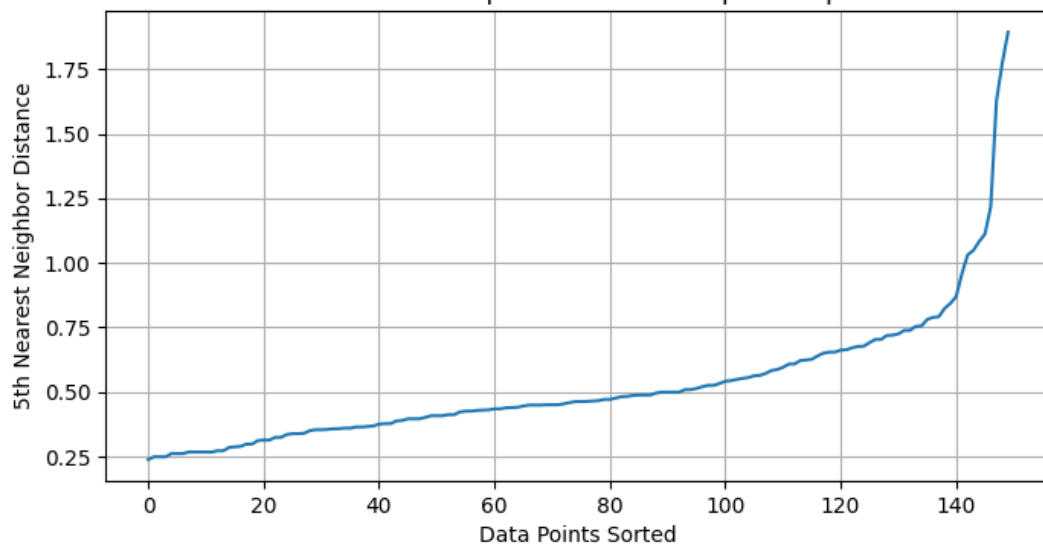
min_samples = 7, Silhouette Score = 0.5216

min_samples = 8, Silhouette Score = 0.5098

min_samples = 9, Silhouette Score = 0.5098

Best min_samples = 7 with silhouette score = 0.5216

K-distance Graph to Determine Optimal eps



Question 5

```
from sklearn.cluster import DBSCAN
import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("Dataset_Day14.csv")
features = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']
X = df[features]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Use DBSCAN (e.g., eps=0.8, min_samples=4)
dbscan = DBSCAN(eps=0.8, min_samples=4)
df['DBSCAN_Cluster'] = dbscan.fit_predict(X_scaled)

# Identify outliers (label -1)
outliers = df[df['DBSCAN_Cluster'] == -1]

# Output statements
print("\n Outlier Detection using DBSCAN")
print(f"Total number of points in dataset: {len(df)}")
print(f" Total outliers detected (cluster label -1): {len(outliers)}")
print("\nOutlier details:")
print(outliers[['Id', 'Species', 'DBSCAN_Cluster']])
```

```
Outlier Detection using DBSCAN
Total number of points in dataset: 150
Total outliers detected (cluster label -1): 4

Outlier details:
   Id  Species DBSCAN_Cluster
41  42  Iris-setosa           -1
109 110  Iris-virginica        -1
117 118  Iris-virginica        -1
131 132  Iris-virginica        -1
```